Stochastic Approximate DP and Discrete-Event Simulation for EV Charging Allocation

Dhrubo Chattoraj

Boston University, Department of Systems Engineering

Final Project — Discrete Event and Hybrid Systems

Abstract

This project attempt to create a stochastic approximation dynamic programming (ADP) and discrete-event simulation (DES) framework to determine the allocation of arriving electric vehicles (EVs) at charging stations, charger types or time-shared multi-port resources, utilizing a lookahead policy derived from a learned value function.

Additionally, the local multi-port charger operation is handled using an Earliest Deadline First (EDF) scheduling rule where multiple vehicles are connected to a time-shared multi-port charger, but only one vehicle can receive power from it at any given time.

The system is coded in Python and tested under several different arrival rates and policies, showing that the DP policy has better performance across all rates of vehicle congestion. The DP policy achieves a lower deadline-miss rate and a higher served fraction than the baselines.

1 Introduction

The rapid growth of battery-powered vehicles, such as Electric Vehicles (EVs), and the lack of sufficient charging infrastructure have created new problems. This gap has already manifested in urban areas, where space for constructing even a single charging station is scarce and expensive. Hence, there is now a need for algorithms and policies to recover more usage and revenue from existing infrastructure, such as dynamic reservation[1, 2], time-of-use (TOU) pricing, and shared hardware. One example is time-shared multi-port charging[3], where a single charging resource can serve multiple EV-side connector ports.

Problem: We have a network of charging stations populated with a heterogeneous mix of solo chargers with at least one multi-port charger in the total network. The EVs arrive as a Poisson process with each EV carrying the following information: (i) an energy requirement e_i, (ii) a departure deadline d_i, and (iii) a minimum acceptable charger power ρ_i. The electricity prices also vary over the day according to a determined time-of-use (TOU) schedule. Two problems arise. At each arrival, a system-level controller must assign the EV to a station-charger pair. If the multi-port charger is selected, a local controller must decide, at each scheduling epoch, which admitted EV receives power.

The objective we want to achieve is the average per-EV cost, which is defined as a weighted sum of waiting time, charging cost, and the deadline-miss penalty.

Model: The system is modeled as a continuous-time discrete event system with state values such as queue lengths, multi-port charger occupancy, remaining energy, and the current pricing regime. The events are in the form of arrivals, service start and completions, admission to a multi-port level, deadline expiration, and changes in the TOU price. The two-level decision problem becomes a discrete-event Markov decision process with a Bellman equation discounted between epochs. Since the state space becomes exceedingly large , finding an exact value iteration solution is infeasible even after discretization, we apply an approximation dynamic programming method at the upper level and a simple scheduling policy at the lower level. At the upper level, a linear value function approximation is trained via temporal difference learning. The lower-level decision problem, however, is solved using the EDF policy.

2 System Model

$\mathcal{S}$ is defined as the set of charging station indices and $\mathcal{L}$ are the available charger types with rated power P_l with base prices π_l. Each station s contains a subset $L_S \subseteq L$ of charger types with $n_{s,l}$ physical chargers of type l.

Some stations have time-shared multi-port chargers, where each charger has k_c admit slots and a type l_c. Charging through this resource has a discounted price of $\pi_{l,c} f_c$ per kWh, where $f_c \in (0, 1)$ is the price factor. This discount is there because the multi-port charger will only draw power when the scheduler activates, interrupting a steady power supply to the EV causing inconvenience.

The EV arrives as a Poisson process with rate λ. Each EV has e_i energy demand, a deadline d_i, and a minimum acceptable charger power ρ_i. Meanwhile, for a the shared charger, the per-EV power is lower as the feeder is time-shared.

Each EV also has preference variables (M_i, D_i, w_i), each representing willingness to pay, maximum travel distance, and cost vs. distance preference, respectively. These are used only for the assignment score (using a one-step lookahead policy) and not for the actual charging.

All parameter values/distributions are given in Appendix A.

2.1 State and Events

For a single regular charger set, two quantities are being tracked. The number of EVs waiting in the queue, denoted by $q_{s,\ell}(t)$, and the set of EVs currently charging, denoted by $B_{s,\ell}(t)$.

For a station s having $n_{s,\ell}$ chargers of type ℓ, then at max, $n_{s,\ell}$ EVs can be serviced at the same time. Hence,

$$|B_{s,\ell}(t)| \leq n_{s,\ell}.$$

Each EV currently being serviced is represented by its remaining energy requirement and the deadline, i.e., (e_j, d_j). For each time-shared multi port charger (s, c), the simulation keeps a track of the occupancy $q_{s,c}(t)$, which is the number of EVs admitted into the multi-port.

Since the multi-port c has k_c available connector slots,

$$|O_{s,c}(t)| \leq k_c.$$

In a regular charger pool where every EV is charging normally, a time-shared multi-port can admit up to k_c EVs into its connection slots while utilizing a single feeder to charge a single vehicle at a time. The EVs already admitted into the shared system but not currently charging still sit in the system. Every time a scheduler event fires, the system scheduler selects which vehicle is receiving power at that moment and changes the order of service as appropriate.

Therefore, the active occupant satisfies.

$$a_{s,c}(t) \in O_{s,c}(t) \cup \emptyset,$$

where $a_{s,c}(t) = \emptyset$ means that the c system is idle and no admitted EV is currently charging. The full system state at time t is therefore

$$X(t) = \left(q_{s,\ell}(t), B_{s,\ell}(t)(s,\ell), qs, c(t), O_{s,c}(t), a_{s,c}(t)_{(s,c)}, p(t)\right). \tag{1}$$

$p(t)$ is the time-of-use multiplier and is deterministic.

For each EV j in the system, the remaining energy $e_j(t)$ and the deadline are also tracked. If j is actively charging on ℓ then its remaining energy is decreasing by $\dot{e}_j(t) = -P_\ell$.

2.2 Decision events and feasible actions

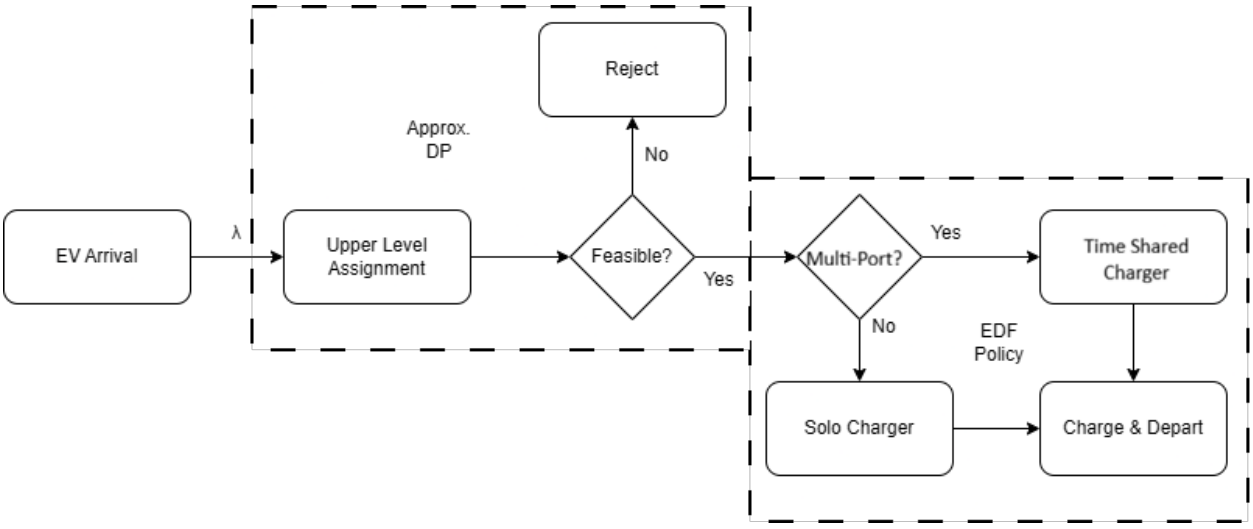

Figure 1: Block diagram of the two-level allocation process.

The model has two layers of decision types as shown in Fig 1 that run at the same simulation time. On each EV's arrival, an upper-level controller assigns EV i to a station along with a charger type and possibly a multi-port charger,

$$u_i \in \mathcal{A}_i(X),$$

with a feasible action set,

$$\mathcal{A}_i(X) = (s, \ell, c) : s \in \mathcal{S},\ \ell \in \mathcal{L}s \cap \mathcal{L}(i),\ c \in \emptyset \cup \mathcal{C}s, \ell. \tag{2}$$

The model also allows the fallback action $u_i = \emptyset$, which corresponds to rejecting the EV. This option is relevant only when the algorithm is unable to meet EV constraints, such as its budget M_i or maximum acceptable distance D_i. The second level is the multi-port scheduling decision. These decisions occur in the multi-port charger by an EV admission, completion of the current session, an occupant switch, or expiry of a scheduling interval. For multi-port charger c at station s, the action set is

$$\mathcal{A}c(Os, c) = O_{s,c} \cup \emptyset,$$

where choosing an EV in $O_{s,c}$ means EV charging, while choosing $\emptyset$ means leaving the multi-port idle.

2.3 Cost Structure

The cost for each EV is written as the sum of three components: the waiting cost, the charging cost, and the deadline-miss penalty.

τ_i^{arr} denotes the arrival time of the EV i, τ_i^{conn} denotes the first time EV i begins receiving power, and τ_i^{dep} is its departure time. The waiting time is

$$W_i = \tau_i^{\text{conn}} - \tau_i^{\text{arr}}. \tag{3}$$

The cost of charging is the integral of the electricity price over the time period during which the EV receives energy:

$$C_i = \int_{\tau_i^{\text{conn}}}^{\tau_i^{\text{dep}}} p(t)\, \pi_{\ell_i}\, \text{rate}(i, \ell_i)\, \mathbf{1}\{i \text{ is active at time } t\}\, dt. \tag{4}$$

Here, $p(t)$ is the time-of-use electricity multiplier, π_{ℓ_i} is the base price of the charger type assigned to EV i, and $\text{rate}(i, \ell_i)$ is the charging rate delivered to the EV.

If an EV leaves before receiving the required charge, there is a deadline miss penalty,

$$D_i^{miss} = \kappa \max\left(0, e_i(\tau_i^{\text{dep}})\right),$$

where κ is the penalty charged per kWh of unmet demand.

The total cost for EV i is then:

$$J_i = w_W W_i + w_C C_i + w_D D_i^{miss}, \qquad \bar{J}(T) = \frac{1}{N_T} \sum_{i=1}^{N_T} J_i. \tag{5}$$

Since $p(t)$ is deterministic, the charging-cost integral can be calculated as a finite sum over the price intervals touched by the session. If a charging session crosses time-of-use boundaries

$$0 = h_0 < h_1 < \cdots < h_m = 24,$$

Then the energy cost is calculated by taking the sum of the contributions from each interval.

We report two cost views. The *cost per served EV* (Table 4) conditions on service and rewards policies that shed demand. The cost per arrival (Table 7) charges every arrival its realized charging cost plus the miss penalty $\kappa \max(0, e_i(\tau_i^{\text{dep}}))$ on unmet energy, and is the metric consistent with the per-EV objective $\bar{J}$ in Eq. (5).

3 Dynamic Programming Formulation

3.1 Discrete-Event Bellman Equation

The system can be viewed as a decision process observed at event times[4]. Let k index the decision events, and let $\Delta_k = t_{k+1} - t_k$ be the random time between two consecutive decision events. At event k, the system is in state X_k, and the controller chooses an action $U_k \in \mathcal{A}(X_k)$. The optimal cost-to-go satisfies the discrete-event Bellman equation

$$V^*(X_k) = \min_{U_k \in \mathcal{A}(X_k)} \mathbb{E}\left[g(X_k, U_k, t_k, \Delta_k) + \gamma^{\Delta_k} V^*(X_{k+1}) \,\middle|\, X_k, U_k\right]. \tag{6}$$

Here, $g(X_k, U_k, t_k, \Delta_k)$ is the cost accumulated over the interval $[t_k, t_{k+1})$. It is consistent with the per-EV cost definition in Eq. (5): waiting cost is accumulated for EVs in the queue, charging cost

is accumulated for EVs actively receiving power, and deadline penalties are added for EVs that depart with unmet energy demand.

$$\begin{aligned} g(X_k, U_k, t_k, \Delta_k) = & \sum_{j \in \text{queue}} \int_{t_k}^{t_{k+1}} w_W \, dt \\ & + \sum_{j} \int_{t_k}^{t_{k+1}} w_C \, p(t) \, \pi_{\ell_j} \, \text{rate}(j, \ell_j) \\ & \qquad \times \mathbf{1}\{\, j \text{ energized at } t \,\} \, dt \\ & + \sum_{j \in \text{depart}} w_D \, D_j^{\text{miss}}. \end{aligned} \tag{7}$$

The factor γ^{Δ_k} discounts future cost according to the amount of real time that passes between events. Equivalently,

$$\gamma^{\Delta_k} = e^{-\beta \Delta_k}, \qquad \beta = -\ln \gamma.$$

We apply a constant discount γ per event rather than scaling by elapsed time, since Δ_k is unknown at assignment. The discount keeps training stable, but the reported metric is the undiscounted average per-EV cost $\bar{J}$ (Eq. (5)). The expectation in Eq. (6) is taken over the random event timing, the next state X_{k+1}, and the attributes of any newly arriving EV.

Solving this equation exactly is impractical, as even after discretizing the state space, the number of states is far too large for a tabular dynamic-programming solution.

For this reason, the implementation uses an approximate two-level approach. The upper level makes EV assignment decisions using a value-function approximation and one-step lookahead. The lower level handles the time-shared multi-port charger scheduling and uses an EDF policy in order to keep the experimental comparison focused on the upper-level assignment policies.

3.2 Upper-level approximation from the Bellman equation

An upper-level decision is to be made whenever a new EV arrives. Starting from Eq. (6), the ideal dynamic-programming rule is to choose the assignment $u_i \in \mathcal{A}_i(X_k)$ that minimizes the present cost incurred plus the future expected cost. If the exact value function $V^*(X)$ was available then the optimal assignment would be chosen by comparing the immediate cost of each feasible assignment with the anticipated value of the next state.

The DP logic would be

$$\text{choose assignment} = \text{immediate assignment cost} + \text{future congestion cost}$$

This is because the cheapest option for the current EV is not always the best option for the system. For example, routing too many EVs to the lower cost chargers may reduce current EVs' costs, but creates long queues and deadline misses later.

Since the exact value function $V(X)$ is complex (Section 3.1), the exact value function is replaced by a linear approximation:

$$V(X) \approx \hat{V}(X; \theta) = \theta^\top \phi(X).$$

Here, $\phi(X)$ is a compact vector which summarizes the main information required for assignment, and θ is a vector of learned weights. The feature vector used in the implementation is

$$\phi(X) = [1,\ \bar{u},\ u_{\max},\ q_{\text{tot}}/10,\ p(t),\ \mathbf{1}h \in [0,6),\ \mathbf{1}h \in [6,12),\ \mathbf{1}h \in [12,18),\ \mathbf{1}h \in [18,24)]\,. \quad (8)$$

In the above vector, $\bar{u}$ is the average busy fraction across pools that are charging EVs, $u_{\max}$ is the maximum busy fraction among all pools, q_{tot} is the total number of EVs waiting in queues, and $p(t)$ is the current time-of-use multiplier. The variable $h = t \bmod 24$ is the hour of the day, and the indicator terms divide the day into four time blocks. These features give only a low-dimensional summary of congestion, queuing, price, and time of day, not the full system state.

Using this approximation, the exact DP assignment rule becomes a one-step lookahead rule:

$$u_i^* \in \arg\min_{u \in \mathcal{A}_i(X_k)} \left[\hat{c}_i(u) + \gamma \hat{V}(X_k'; \theta)\right]. \quad (9)$$

This equation is the actual upper-level policy used in the simulation. The first term, $\hat{c}_i(u)$, estimates the immediate cost of assigning EV i to candidate assignment u. The second term, $\gamma \hat{V}(X_k'; \theta)$, estimates the future cost after that assignment is made. The state X_k' is a simple post-decision state, i.e., it represents the approximate state after assigning the EV, but before the next random event occurs.

The immediate assignment cost[1] is computed as

$$\hat{c}_i(u) = w_i \frac{\hat{C}_i(u)}{M_i} + (1 - w_i)\frac{d_{is}}{D_i} + \kappa\, \mathbf{1}\left[\hat{t}_{is} + \hat{W}(u) + \frac{e_i}{P_\ell} > d_i - \tau_i^{\text{arr}}\right] \quad (10)$$

The score has three parts. The first term is the estimated charging cost $\hat{C}_i(u)$ normalized by the EV's willingness-to-pay budget M_i. The second term is the travel distance d_{is} to station s, normalized by the EV's maximum acceptable distance D_i. The weight w_i controls how much the EV prioritizes charging cost relative to distance. The third term is the deadline-risk penalty. It checks whether the estimated travel time, waiting time, and charging time exceed the EV's available time before its deadline. If the assignment is likely to miss the deadline, the penalty κ is added.

The parameter vector θ is trained from simulated trajectories using temporal-difference learning[5]. After observing a transition from feature vector ϕ_k to ϕ_{k+1} with cost c_k, the update is

$$\theta \leftarrow \theta + \eta \left(c_k + \gamma \theta^\top \phi_{k+1} - \theta^\top \phi_k\right) \phi_k. \quad (11)$$

The expression inside parentheses is the temporal-difference error. It compares the current value estimate $\theta^\top \phi_k$ with the observed cost plus the estimated value of the next state. The learning rate η controls how strongly the weights are updated. During training, an ϵ-greedy rule is used so that the policy sometimes explores assignments other than the one currently predicted to be best.

Thus, the upper-level policy is an approximate version of the Bellman assignment rule. The exact value function $V^*(X)$ is replaced by the learned approximation $\hat{V}(X; \theta)$, and the expectation over all possible future events is replaced by a cheap one-step post-decision estimate. This makes the policy computationally feasible inside the discrete-event simulator while still allowing it to account for future congestion.

3.3 Multi-port level scheduling

The lower-level decision problem arises only when an electric vehicle (EV) is assigned to a multi-port charger. Unlike in a regular charger, where an EV that begins charging will be continuously powered by the charger, the discounted charger allows for multiple EVs to be admitted, while only one of the admitted EVs is being charged at any point in time. The scheduler for the multi-port charger, therefore, must decide which of the admitted EVs must receive power in each scheduling epoch. For a multi-port charger c at station s, the feasible scheduling actions are:

$$\mathcal{A}c(Os, c) = O_{s,c} \cup \emptyset,$$

where selecting an EV charges it while selecting $\emptyset$ leaves the charger idle. In the simulation, the multi-port scheduler uses an EDF policy, whose pseudo-code is available in Appendix D.

4 Implementation and Results

The simulation compares the approximate-DP assignment policy, DPUpper, against four heuristic upper-level assignment policies: GreedyCost, GreedyDelay, WeightedGreedy and LeastLoaded. A concise definition of each policy is given in Appendix B. The values computed are reported in Appendix C.

Each policy–arrival-rate pair is run for two random seeds over 24 simulated hours, giving $5 \times 4 \times 2 = 40$ runs.

4.1 Experimental Design and Results

As seen in Fig 2 GreedyDelay, LeastLoaded, and DPUpper achieve the highest served fraction, while GreedyCost and WeightedGreedy serve fewer EVs as they end up overloading cheaper, slower chargers. DPUpper gives the lowest deadline-miss rate across all tested loads, with a deadline-miss rate of 0.346 at $\lambda = 10$, and it retains this advantage even at the highest tested load, $\lambda = 20$.

GreedyCost and WeightedGreedy have the lowest average cost per EV served, but this is misleading as they serve fewer EVs overall and allow deadline failures. On cost per arrival (Table 7), which charges each arrival a penalty for unmet energy, the ranking inverts: GreedyCost becomes the most expensive policy while DPUpper becomes the cheapest at every load. DPUpper, GreedyDelay, and LeastLoaded also achieve shorter charging times since they use higher-power chargers more often. The average EV charging costs in DPUpper policy are the highest among all the policies. This is because of how the total charging costs are constructed, in Eq. 4. While DPUpper might pay higher costs for each EV's charging in total, the number of missed charging events has reduced, achieving a much larger satisfied demand portion and instead focusing less on minimizing its charging costs as an isolated metric.

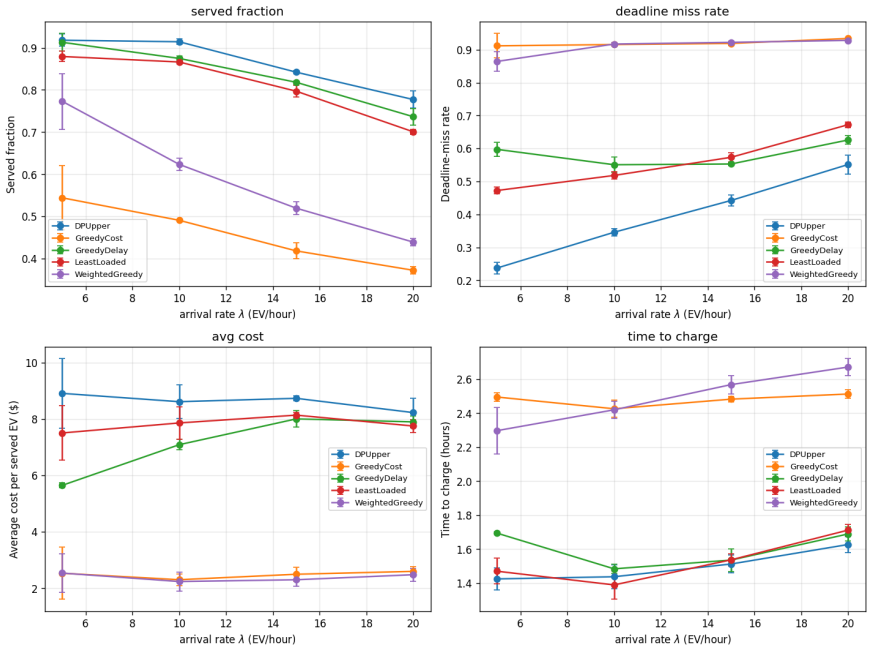

Figure 2: Four metrics versus arrival rate λ for five policies. Error bars show one standard deviation across two seeds.

Overall, the approximate-DP policy captures useful congestion information and gives a consistent improvement over the myopic baselines across the tested demand levels, with the small network size and short training horizon as the main remaining limitations.

4.2 Multi Port Charger Usage

The fraction of arriving EVs assigned to the time-shared charger by each policy is as shown in Figure 3. The main pattern is that DPUpper routes the largest share of EVs to the multi-port at low load ($\lambda = 5$), but its multi-port usage largely falls as the arrival rate rises, while LeastLoaded's stays high; LeastLoaded exceeds DPUpper at $\lambda = 15$ and $\lambda = 20$.

The results show that the value-function lookahead treats the time-shared charger as relief capacity rather than a fixed preference. When the network is lightly loaded and deadlines have slack, it routes a large share to the multi-port to exploit its discounted price, but as the port fills it backs off, since loading an already busy port is penalized. The charger's multiple admission slots and discounted price make it an attractive option for absorbing congestion, but only while the time-shared feeder can still meet deadlines.

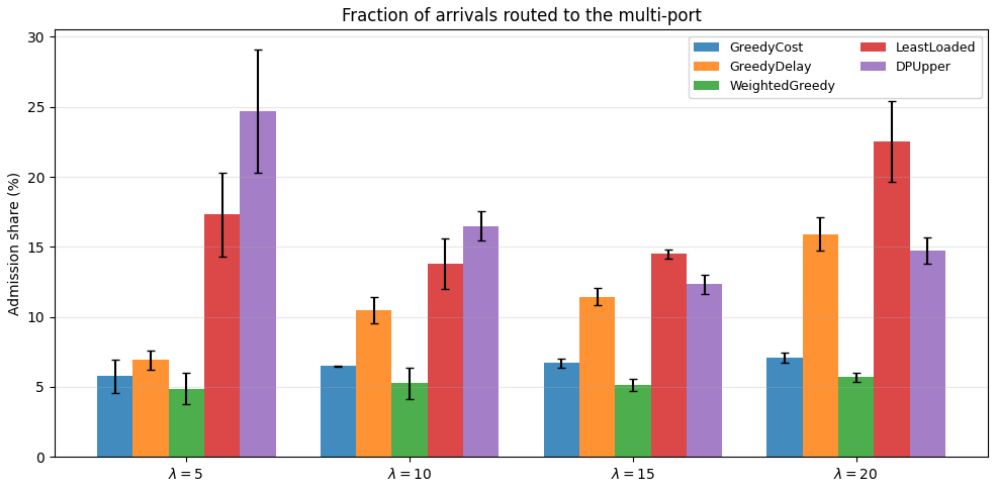

Figure 3: Fraction of arrivals routed to the shared charger by each upper-level assignment policy. DPUpper assigns the largest share of EVs to the multi-port at low load but reduces its usage as congestion increases. Error bars show one standard deviation across two random seeds.

5 Conclusion

This project developed a stochastic approximate DP discrete event system for EV charging allocation with a multi-port time-sharing policy. The upper-level DP assigns the arriving EVs to stations, charger types, or the multi-port charger using an approximate value-function lookahead, while the lower-level scheduler uses an EDF policy, which keeps the multi-port scheduling rule fixed across comparisons.

The main results are that DPUpper, the approximate-DP policy, achieves the lowest deadline-miss rate and the highest served fraction across all tested arrival rates, including the highest load $\lambda = 20$, which indicates that the value-function lookahead captures useful congestion information. The improvement over the myopic baselines is consistent rather than large.

Another finding is that DPUpper adapts its use of the time-sharing charger to the load: it routes the largest share of arrivals to the multi-port at low congestion but reduces this share as the network fills, treating the multi-port as relief capacity rather than a fixed preference or a mere fallback.

The main limitations are the small network size, the simplified single EV charging multi-port model, the use of a simple EDF scheduler, and the lack of any battery characteristics. Future work can extend experiments to a bigger network, train the model over longer horizons, and enhance the complexity of the multi-port model.

References

[1] C. G. Cassandras and Y. Geng, "Optimal dynamic allocation and space reservation for electric vehicles at charging stations," *IFAC Proceedings Volumes*, vol. 47, no. 3, pp. 4056–4061, 2014. 19th IFAC World Congress, Cape Town, South Africa.

[2] Y. Geng and C. G. Cassandras, "A new smart parking system based on resource allocation and reservations," *IEEE Transactions on Intelligent Transportation Systems*, vol. 14, no. 3, pp. 1129–1139, 2013.

[3] B. Mukherjee and F. Sossan, "Optimal planning of single-port and multi-port charging stations for electric vehicles in medium voltage distribution networks," 11 2021.

[4] D. P. Bertsekas, *Dynamic Programming and Optimal Control*. Belmont, MA: Athena Scientific, 4th ed., 2017.

[5] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA: MIT Press, 2nd ed., 2018.

[6] U.S. Department of Transportation, "Charger types and speeds." Rural EV Toolkit, https://www.transportation.gov/rural/ev/toolkit/ev-basics/charging-speeds, 2025. Accessed: 2026-05-30.

Appendix

Appendix A: Variables and Parameters

The simulation uses the three charging stations

$$\mathcal{S} = \{0, 1, 2\}$$

and four charger types[6]

$$\mathcal{L} = \{L2s, L2f, DCm, DCh\}.$$

The rated powers are

$$P_l \in \{7, 11, 50, 150\}kW$$

and base prices be

$$\pi_l = \{0.12, 0.15, 0.30, 0.45\}\$/\text{kWh}.$$

The multi-port charging is offered at a discounted price of $\pi_{l,c} f_c$ where $f_c = 0.7$.

EVs arrive according to a homogeneous Poisson process with rate λ. In this experiment, the values are

$$\lambda \in \{5, 10, 15, 20\}$$

The sampled EV attributes are

$$e_i \sim \mathcal{N}_{[10,60]}(35, 12^2), \qquad d_i - \tau_i^{\text{arr}} \sim \mathcal{U}(0.5, 4.0),$$
$$M_i \sim \mathcal{U}(5, 50), \qquad D_i \sim \mathcal{U}(1, 20), \qquad w_i \sim \mathcal{U}(0, 1),$$

and

$$\rho_i \in \{0, 7, 50\} \text{ kW}$$

with probabilities $\{0.6, 0.3, 0.1\}$.

Unless otherwise stated, the experiment uses equal weights

$$w_W = w_C = w_D = 1$$

Appendix B: Policy Comparison

Table 1 summarizes the policies used in the simulation. The first five policies control the upper-level assignment of arriving EVs. EDF is used as the fixed multi-port scheduler.

Policy	**Decision Level**	**Rule**
GreedyCost	Upper-level	Assigns the EV to the charger option with the lowest estimated charging cost. This favors cheaper chargers but may overload slower resources.
GreedyDelay	Upper-level	Assigns the EV to the option with the lowest estimated waiting and charging delay. This prioritizes faster service and deadline feasibility.
WeightedGreedy	Upper-level	Uses the EV preference weight w_i to trade off normalized charging cost and travel distance. It has user preference but does not account for future congestion.
LeastLoaded	Upper-level	Assigns the EV to the feasible charger with the lowest current utilization or queue burden. This spreads demand across the entire network.
DPUpper	Upper-level	Uses one-step lookahead with a learned value-function approximation $\hat{V}(X;\theta)$. The assignment tries to minimize immediate estimated cost plus approximate future congestion cost.
EDF	Lower-level	Earliest-deadline-first scheduling. Among EVs already admitted to a multi-port, EDF energizes the EV with the smallest remaining time to deadline.

Table 1: Summary of all policies.

Appendix C: Results

Table 2: Comparison of policies by served fraction

Policy	$\lambda = 5$	$\lambda = 10$	$\lambda = 15$	$\lambda = 20$
DPUpper	0.918	0.914	0.842	0.777
GreedyDelay	0.913	0.875	0.818	0.737
LeastLoaded	0.879	0.866	0.797	0.701
WeightedGreedy	0.773	0.624	0.520	0.440
GreedyCost	0.544	0.491	0.419	0.373

Table 3: Comparison of policies by deadline-miss rate

Policy	$\lambda = 5$	$\lambda = 10$	$\lambda = 15$	$\lambda = 20$
DPUpper	0.237	0.346	0.442	0.551
GreedyDelay	0.597	0.551	0.553	0.626
LeastLoaded	0.473	0.518	0.573	0.672
WeightedGreedy	0.864	0.917	0.922	0.928
GreedyCost	0.912	0.916	0.919	0.934

Table 4: Comparison of policies by average cost per EV served ($)

Policy	$\lambda = 5$	$\lambda = 10$	$\lambda = 15$	$\lambda = 20$
DPUpper	8.909	8.614	8.733	8.227
GreedyDelay	5.649	7.090	8.004	7.899
LeastLoaded	7.506	7.861	8.137	7.752
WeightedGreedy	2.540	2.241	2.304	2.483
GreedyCost	2.535	2.306	2.499	2.601

Table 5: Comparison of time to charge (hours) by upper-level policy.

Policy	$\lambda = 5$	$\lambda = 10$	$\lambda = 15$	$\lambda = 20$
DPUpper	1.425	1.437	1.512	1.627
GreedyDelay	1.694	1.484	1.536	1.689
LeastLoaded	1.470	1.389	1.537	1.712
WeightedGreedy	2.298	2.420	2.569	2.672
GreedyCost	2.496	2.426	2.483	2.513

Table 6: Fraction of Arrivals routed to a multi-port

Policy	$\lambda = 5$	$\lambda = 10$	$\lambda = 15$	$\lambda = 20$
GreedyCost	5.76 ± 1.19	6.51 ± 0.04	6.72 ± 0.33	7.09 ± 0.34
GreedyDelay	6.93 ± 0.68	10.48 ± 0.96	11.44 ± 0.61	15.91 ± 1.21
WeightedGreedy	4.88 ± 1.10	5.25 ± 1.10	5.15 ± 0.43	5.69 ± 0.30
LeastLoaded	17.32 ± 3.00	13.77 ± 1.80	14.49 ± 0.35	22.52 ± 2.87
DPUpper	24.69 ± 4.37	16.49 ± 1.04	12.32 ± 0.68	14.76 ± 0.95

Table 7: Cost per arrival ($): charging cost plus deadline-miss penalty $\kappa \max(0, e_i(\tau_i^{\text{dep}}))$ with $\kappa = 5\,\$/\text{kWh}$(averaged over all arrivals).

Policy	$\lambda = 5$	$\lambda = 10$	$\lambda = 15$	$\lambda = 20$
DPUpper	**49.28**	**61.73**	**79.92**	**96.21**
GreedyDelay	80.44	76.46	86.56	100.37
LeastLoaded	67.85	73.78	86.91	104.07
WeightedGreedy	123.08	136.31	144.72	150.58
GreedyCost	140.86	143.63	147.67	152.40

Appendix D: Pseudocode

Algorithm 1 Upper-level one-step lookahead assignment (DPUpper)

Require: arriving EV i with attributes $(e_i, d_i, M_i, D_i, w_i, \rho_i)$ and arrival time τ_i^{arr}; system state X; weights θ; discount γ; deadline penalty κ

Ensure: assignment $u_i^\star \in \mathcal{A}_i(X) \cup \{\varnothing\}$

$\mathcal{A}_i(X) \leftarrow \{(s, \ell, c) :\ \text{dist}(i, s) \le D_i,\ \ell \in L_s \cap L(i),\ c \in \{\varnothing\} \cup C_{s,\ell}\}$ ▷ Eq. (2)

if $\mathcal{A}_i(X) = \varnothing$ **then**

  return $\varnothing$ ▷ reject: no feasible station/charger

end if

best $\leftarrow +\infty$, $u_i^\star \leftarrow \varnothing$

for all $u = (s, \ell, c) \in \mathcal{A}_i(X)$ **do**

  $\hat{C}_i(u) \leftarrow$ TOU integral of delivering e_i at (ℓ, c) ▷ Section 2.3; multi-port applies factor f_c

  if $\hat{C}_i(u) > M_i$ **then**

    continue ▷ over budget

  end if

  $\hat{c}_i(u) \leftarrow w_i \dfrac{\hat{C}_i(u)}{M_i} + (1 - w_i) \dfrac{\text{dis}}{D_i} + \kappa\, \mathbf{1}\Big\{\hat{t}_{is} + \hat{W}(u) + \frac{e_i}{P_\ell} > d_i - \tau_i^{\text{arr}}\Big\}$ ▷ Eq. (10)

  form post-decision state $X'(u)$; $\hat{V}(X'(u); \theta) \leftarrow \theta^\top \phi\big(X'(u)\big)$

  score $\leftarrow \hat{c}_i(u) + \gamma\, \hat{V}\big(X'(u); \theta\big)$ ▷ Eq. (9)

  if score $<$ best **then**

    best $\leftarrow$ score, $u_i^\star \leftarrow u$

  end if

end for

return $u_i^\star$

Algorithm 2 TD(0) training of the value-function weights θ

Require: station generator, attribute sampler, TOU schedule; training rate λ_{tr} and horizon T; epochs N; exploration rate ϵ; step size η; discount γ
Ensure: trained weights θ
$\theta \leftarrow \mathbf{0}$
for epoch $= 1, \ldots, N$ **do**
  simulate one episode of length T at rate λ_{tr} under the ϵ-greedy policy:
    with prob. ϵ pick a uniformly random feasible action; otherwise apply DPUpper(θ)
  harvest transitions $\{(\phi_k, c_k, \phi_{k+1})\}$, where $\phi_k = \phi(X_k)$ and c_k is the realized per-EV cost
  for all $(\phi_k, c_k, \phi_{k+1})$ **do**
    $\delta \leftarrow c_k + \gamma\, \theta^\top \phi_{k+1} - \theta^\top \phi_k$ ▷ temporal-difference error
    $\theta \leftarrow \theta + \eta\, \delta\, \phi_k$ ▷ Eq. (11)
  end for
end for
return θ

Algorithm 3 EDF scheduling rule

Require: Connected EV set $O_{s,c}$, current time t, deadlines $\{d_i : i \in O_{s,c}\}$
Ensure: Selected EV $a^\star \in O_{s,c} \cup \{\emptyset\}$
if $O_{s,c} = \emptyset$ **then**
  return $\emptyset$
end if
$a^\star \leftarrow \arg\min_{i \in O_{s,c}} (d_i - t)$
return $a^\star$

Appendix E: AI Usage Disclosure

AI tools were used during this project. Claude (Opus 4.7) was used for LaTeX editing, help with code debugging and reviewing of output, and grammar/syntax checking. The modeling, DES formulation, DP and RL policies were my own. The report was written by me in Overleaf.